Web server loading

Here is the fully updated Markdown note, ready for your Obsidian vault. Step 8 has been expanded to cover both the single-machine "Hosts File" method and the multi-machine "BIND9 DNS" method, keeping the formatting and "🧠 Why" explanations consistent throughout.

---

tags:

- webserver
- apache
- ubuntu
- linux

aliases:

- Apache Setup Guide
- Apache Web Server (httpd)

Apache Web Server Setup

Apache (often referred to as the `httpd` daemon in some Linux distributions) is a popular open-source web server.

1. Checking Server Load / Ports

To check listening ports and see if a web server is currently running, use:

Bash

```
ss -tlpn
```

🧠 **Why are we doing this?**

Before installing a web server, it's good to check if port 80 (the default port for web traffic) is already being used by another program. If another web server (like Nginx) is already running, Apache will fail to start because two programs can't listen on the same exact port at the same time.

2. Installation

To install Apache on Ubuntu:

Bash

```
sudo apt install apache2
```

🧠 **Why are we doing this?**

This command downloads and installs the Apache web server software. Once installed, it runs as a background service (daemon) waiting for people to ask for web pages.

Verification

Open your web browser.

Type `localhost` (or the server's IP address) in the address bar and press Enter.

You should see the Apache2 Ubuntu Default Page.

The Default Page

The page you see initially is simply a placeholder `index.html` file provided by Ubuntu to confirm Apache is working correctly. It also contains basic configuration and directory structure information.

3. Managing Web Content

By default, Apache serves files from the `/var/www/html/` directory.

🧠 **Why `/var/www/html/`?**

A web server shouldn't let people on the internet browse your entire computer. It needs a specific, locked-down folder that is deemed "safe for the public." By default, Apache uses `/var/www/html/` as its public "storefront."

Navigating to the Web Root

Bash

```
cd /var/www/html/
```

Backing up the Default Page

Instead of deleting the default page, it's good practice to rename (move) it so you have a backup:

Bash

```
sudo mv index.html index.html.orig
```

🧠 **Why are we doing this?**

The default page actually has a lot of useful documentation about how Apache is set up on Ubuntu. By renaming it to `.orig` (original), we hide it from the web server (which only looks for exactly `index.html`) but keep it on our drive just in case we want to read it later.

Creating Your Own Page

Create a new `index.html` file:

Bash

```
sudo nano index.html
```

Note: Whatever HTML you write and save in this file will now be displayed in your browser at `localhost` upon refreshing the page.

4. Setting up a Virtual Site (VirtualHosts)

Virtual Hosts allow you to run multiple websites on a single Apache server.

🧠 **Why use Virtual Hosts?**

Imagine you own `myblog.com` and `mystore.com`, but you only have one physical server. Virtual Hosts let Apache look at the incoming request, say "Ah, you're asking for the blog," and serve files from a completely different folder than if the user asked for the store.

Navigate to Apache Configuration

Apache configuration files are stored in `/etc/apache2/`.

Bash

```
cd /etc/apache2
```

Create a New Site Configuration

Copy the default site configuration file (`000-default.conf`) to create a template for your new site:

Bash

```
sudo cp sites-available/000-default.conf sites-available/mynewsite.conf
```

🧠 **Why are we doing this?**

Writing a configuration file from scratch is tedious and error-prone. The default configuration file already has the correct syntax and structure. Copying it gives us a perfectly working template that we just need to tweak.

Edit the New Configuration

Open the newly copied configuration file to adjust your site's settings:

Bash

```
sudo nano sites-available/mynewsite.conf
```

Inside this file, you need to update a few key directives. It should look something like this:

Apache

```
<VirtualHost *:80>
    ServerAdmin webmaster@localhost
    ServerName mynewsite.local
    DocumentRoot /var/www/mynewsite

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

🧠 **What do these settings mean?**

- `VirtualHost *:80`: Listen on port 80 (standard HTTP) for all incoming connections.
- `ServerName`: The actual domain name someone types in to reach this specific site.
- `DocumentRoot`: The folder where the files for this specific site are stored.

5. Create the Directory Structure

Now that we told Apache to look for our site at `/var/www/mynewsite` (in the `DocumentRoot` above), we need to actually create that folder.

Bash

```
sudo mkdir -p /var/www/mynewsite
```

Next, assign ownership of the directory so you can edit files inside it without needing `sudo` every time:

Bash

```
sudo chown -R $USER:$USER /var/www/mynewsite
```

🧠 **Why change ownership?**

Directories created with `sudo` are owned by the `root` administrator. If you don't change the ownership to your regular user (`$USER`), you'll be blocked every time you try to save a new web page or upload images via FTP/SFTP.

Create a simple test page for your new site:

Bash

```
nano /var/www/mynewsite/index.html
```

(Add some basic HTML like `<h1>Welcome to My New Site!</h1>` and save).

6. Enable the New Site

Apache comes with built-in tools to enable and disable virtual hosts.

Enable your new site using the `a2ensite` (Apache 2 Enable Site) command:

Bash

```
sudo a2ensite mynewsite.conf
```

🧠 **Why are we doing this?**

Apache stores configurations in `sites-available`, but it doesn't automatically use them all (you might have test sites or inactive sites in there). The `a2ensite` command tells Apache, "I'm ready for this site to go live." (It technically creates a shortcut to your file in a folder called `sites-enabled`).

(Optional) Disable the default site if you only want your new site to load:

Bash

```
sudo a2dissite 000-default.conf
```

7. Test and Restart Apache

Before restarting the web server, it is best practice to test your configuration files for any syntax errors:

Bash

```
sudo apache2ctl configtest
```

(You should see `Syntax OK`. If not, review your `.conf` file for typos).

🧠 **Why are we doing this?**

If you made a typo (like forgetting a `>` bracket) in your config file, restarting Apache will cause the whole web server to crash and refuse to start back up, taking all your websites offline. Testing prevents this.

Finally, reload Apache to apply the changes:

Bash

```
sudo systemctl reload apache2
```

🧠 **Restart vs Reload?**

`reload` is better than `restart`. A full restart kills the server and boots it back up, meaning anyone currently downloading a file from your site gets disconnected. `reload` gracefully finishes current connections while applying the new settings in the background.

8. Network & Local Testing (Hosts File vs. DNS Server)

Because `mynewsite.local` (or `mynewsite.zhjlab.bd`) isn't a real registered domain name on the public internet, computers won't naturally know how to find it. You must map the domain to your server's IP address.

Option A: The "Local" Fix (Single PC via Hosts File)

If you only need to view the site on the exact machine where it's hosted:

Edit your local hosts file:

Bash

```
sudo nano /etc/hosts
```

Add the following line right under `127.0.0.1 localhost`:

Plaintext

```
127.0.0.1   mynewsite.local
```

🧠 **Why edit the hosts file?**

The `hosts` file is like your computer's personal address book. Before your computer asks the global internet where a domain is, it checks this file first. By adding this line, you are tricking your computer into pointing that fake domain directly back to your own machine so you can test your new site locally.

Option B: The "Network" Fix (Multiple PCs via BIND9 DNS)

If you want other devices on your local network (like your phone or a friend's laptop) to access the site, editing the `hosts` file on every single device is impractical. Instead, add an entry to your local DNS server (like BIND9).

1. **Edit the Zone File:** Open the DNS zone file for your lab domain (e.g., `zhjlab.bd`):

   Bash
   ```
   sudo nano /etc/bind/db.zhjlab.bd
   ```
2. **Add an A Record:** Point the new site name to your Apache server's IP address (e.g., `192.168.1.10`):

   Plaintext
   ```
   ; --- Web Server Entries ---
   mynewsite   IN  A   192.168.1.10
   ```
3. **Update & Restart:** Increment the Serial Number in the SOA record of the zone file, then restart BIND9:

   Bash

```
sudo systemctl restart bind9
```

🧠 **Why use a DNS Server?**

Instead of every device needing a manually updated personal address book, they all query your central BIND9 server. As long as the other devices set their network's Primary DNS Server to your Ubuntu server's IP (`192.168.1.10`), they can seamlessly access `http://mynewsite.zhjlab.bd` in their browsers, and Apache will serve the correct files!

Would you like me to generate some example `curl` commands to quickly test the HTTP responses of your new sites directly from the terminal?